

generate_agent_prefix_commands.sh

Revision: 2 **Last modified:** 2026-07-14T22:05:00Z **Authority:** constitution §11.4.140 (universal action-prefix system) + §11.4.202 (reporting directives) + §11.4.28 (decoupling — the registry is data, not code) **Maintainer:** constitution submodule (inherited by reference per §11.4.177 / §11.4.28(B)) **Scope:** §11.4.18 companion doc for `constitution/scripts/generate_agent_prefix_commands.sh`

Overview

`generate_agent_prefix_commands.sh` turns `actions/registry.yaml` — the single source of truth — into the **LAYER-2 slash commands** for every supported CLI agent. It is the mechanism that makes "adding a directive is a registry row, never a code change" literally true: add the row, re-run this generator, and the directive appears on every agent.

For each registered action it emits **two** command files per agent — the bare form and a `default-` alias:

Agent	Files emitted	Argument token
Claude Code	<code>plugins/helix/commands/<name>.md</code> , <code>default-<name>.md</code>	<code>\$ARGUMENTS</code>
Gemini CLI	<code>actions/generated/gemini/<name>.toml</code> , <code>default-<name>.toml</code>	<code>{{args}}</code>
Qwen Code	<code>actions/generated/qwen/<name>.toml</code> , <code>default-<name>.toml</code>	<code>{{args}}</code>
Codex CLI	<code>actions/generated/codex/<name>.md</code> , <code>default-<name>.md</code>	<code>\$ARGUMENTS</code>

It also **regenerates two READMEs** so the published inventory can never drift from the registry: `plugins/helix/README.md` (the Claude Code command table + the conflict rule) and `actions/generated/README.md` (per-agent install targets + the action count).

Claude Code's commands are written **straight into the plugin tree** — the plugin *is* the delivery mechanism, so there is no copy step and no second source of truth.

Why a `default-<name>` alias exists: `::` is not a legal character in a Gemini / Qwen / Codex command name (the name *is* the filename), so those agents cannot express the §11.4.140 form-4 `/PREFIX::ACTION` escape literally. `default-<name>` maps to the **identical expansion**, giving every agent a collision-free invocation. On Claude Code the native plugin namespacing `/helix:<name>` is the form-4 escape, and `/default-<name>` is generated as well, for parity.

System-level narrative: `docs/actions/REFERENCE.md` .

Prerequisites

- `bash` (`set -euo pipefail`), and `scripts/action_prefix_lib.sh` (sourced).
- `python3` **preferred** for reading per-action metadata (`summary` , `slash_conflicts`) and for the library's YAML parse. Without `python3` / `PyYAML` the library falls back to its `awk` reader for names and expansions; the `apx_gen_field` metadata helper then yields empty values and the generator substitutes a **generic** summary and assumes **no** collisions. That degradation is silent by design (non-fatal), so run the generator on a host with `python3` + `PyYAML` if you want the conflict comments and real summaries.
- A registry that passes `apx_validate_registry` — the generator **aborts** (exit 1) otherwise, rather than emitting half-correct commands.

Usage examples

Regenerate every agent's commands from the default registry

```
bash constitution/scripts/generate_agent_prefix_commands.sh
```

Use an alternate registry (e.g. a consumer-owned one, §11.4.28)

```
HELIX_ACTION_REGISTRY=/path/to/registry.yaml \
```

```
bash constitution/scripts/generate_agent_prefix_commands.sh
```

Typical: add a registry row, regenerate, re-wire

```
$EDITOR constitution/actions/registry.yaml
```

```
bash constitution/scripts/generate_agent_prefix_commands.sh
```

```
bash constitution/scripts/install_cli_agent_plugins.sh
```

Output (stderr):

```
generate_agent_prefix_commands: generated 5 action(s) under .../actions/  
generated
```

Edge cases

Case	Behaviour
Registry fails validation	<code>generate_agent_prefix_commands</code> : registry failed validation; aborting , exit 1 . Nothing is written.
An action has no expansion	<code>WARN</code> : no expansion for <code><NAME></code> on stderr; that action is skipped ; the rest are generated. (Validation normally catches this first.)
<code>summary</code> : absent	Falls back to <code>Action-prefix <NAME></code> : run the task with the registered expansion applied.
<code>slash_conflicts</code> : non-empty	The bare command file gets an explicit <code>CONFLICT</code> comment naming the host command and the two safe invocations (<code>/helix:<name></code> , <code>/default-<name></code>), and the plugin README row is rendered as a COLLIDES warning. The plugin never silently shadows a host command.
A <code>summary</code> containing <code> </code> (e.g. <code>{Bug Feature Task}</code>)	Escaped before it is placed in the markdown table cell, so the README table cannot break.
A <code>description</code> containing <code>"</code> or <code>\</code>	Escaped in the YAML frontmatter, so a stray quote cannot break the command file's frontmatter.
Re-run with no registry change	Deterministic — byte-identical files. Safe to re-run at any time; files are overwritten, never appended.
An action removed from the registry	Its command files are not deleted (the generator only writes). Remove the stale <code><name>.md</code> / <code><name>.toml</code> files by hand.

Honest boundary (§11.4.6): the generator only **writes** files. It writes Claude Code's commands straight into the plugin tree, and the Gemini / Qwen / Codex files into `actions/generated/<agent>/` — it does not install anything into `.gemini/commands/` , `.qwen/commands/` or `prompts/` . That wiring is `install_cli_agent_plugins.sh` 's `link_agent_commands()` step, which

symlinks them (never copies, never clobbers a hand-written file). Run the generator alone and the new command exists but is not yet linked into a newly-added agent directory; run the installer (which calls the generator first) and it is.

The generator also does not embed each action's `rules:` into the command files — `rules` are LAYER-1 material, read by the agent from the registry/carrier block. Only `expansion` (plus `summary` and `slash_conflicts` metadata) reaches the command files.

Internal behaviour

```
source scripts/action_prefix_lib.sh
apx_validate_registry || abort(1)

for NAME in $(apx_list_actions):
    expansion = apx_lookup_expansion NAME          # library (python3 →
awk fallback)
    summary   = apx_gen_field NAME summary         # python3-only;
generic fallback
    conflicts = apx_gen_field NAME slash_conflicts # python3-only; ""
fallback
    lc        = lowercase(NAME)

    # form 3 – the bare command
    apx_emit_claude_md plugins/helix/commands/ $lc NAME $expansion
$summary $conflicts
    apx_emit_toml      actions/generated/gemini/ $lc ...
    apx_emit_toml      actions/generated/qwen/   $lc ...
    apx_emit_codex_md  actions/generated/codex/  $lc ...

    # form 4 – the collision-free `default-` alias (identical expansion)
    apx_emit_*          ... default-$lc "DEFAULT::NAME" $expansion ...

    accumulate a README row (COLLIDES variant when $conflicts non-empty)

rewrite plugins/helix/README.md          (command table + conflict rule +
install pointer)
rewrite actions/generated/README.md      (per-agent install-target table +
action count)
```

Emitters: `apx_emit_claude_md` (YAML frontmatter `description: + argument-hint: , a DO NOT EDIT` banner naming the source row, the optional `CONFLICT` comment, the expansion, then `$ARGUMENTS`); `apx_emit_toml` (a `description = ...` key and a `prompt = ""...{{args}}""` block); `apx_emit_codex_md` (HTML-comment banner, the expansion, then `$ARGUMENTS`).

No network, no state, no side effects outside `plugins/helix/` and `actions/generated/` .

Related scripts

Script	Relationship
<code>scripts/action_prefix_lib.sh</code>	The engine: registry validation, action listing, expansion lookup. Sourced by this generator.
<code>in-stall_cli_agent_plugins.md</code>	Calls this generator in its step 1, then wires the plugin + skills.
<code>scripts/in-stall_action_prefix.sh</code>	Also calls this generator, and registers the <code>UserPromptSubmit</code> hook.
<code>scripts/post_update_hook.sh</code>	Reaches this generator indirectly, via <code>in-stall_cli_agent_plugins.sh</code> , on every constitution pull.
<code>actions/registry.yaml</code>	The input — the single source of truth.

Last verified

2026-07-14 — read against `scripts/generate_agent_prefix_commands.sh` at constitution HEAD; emitters, fallbacks, README regeneration, conflict rendering and escaping confirmed line by line; output cross-checked against the generated `plugins/helix/commands/bug.md` and `actions/generated/gemini/bug.toml` .